

Entrega: A entrega da resolução é realizada na página da disciplina no Moodle juntando, num único ficheiro compactado, o código fonte, o Makefile e outros materiais.

Objetivos: Familiarização com o ambiente UNIX/LINUX; Conceção de programas baseados no paradigma cliente/servidor utilizando *sockets* como mecanismo de comunicação entre processos; Conceção de programas concorrentes com base em múltiplas tarefas; Sincronismo entre múltiplas tarefas em POSIX; Utilização de sinais UNIX; Consolidação da programação ao nível de sistema.

I. Resolva as seguintes propostas de exercícios

- Desenvolva o mecanismo de sincronismo `countdown_t` que permite que uma ou mais tarefas esperem que um conjunto de tarefas as sinalizem. O mecanismo é iniciado com um valor inteiro maior que zero (número de tarefas que devem realizar a operação `countdown_down()`). As tarefas que usam a função `countdown_wait()` bloqueiam até que o valor do mecanismo chegue a zero. As tarefas que chamam a primitiva `countdown_down()` não bloqueiam e o valor do mecanismo é decrementado. Quando o valor do mecanismo chegar a zero, todas as tarefas em espera são desbloqueadas e as chamadas subsequentes ao `countdown_wait()` retornam de imediato com a indicação de erro. A Figura 1 ilustra o comportamento do mecanismo.

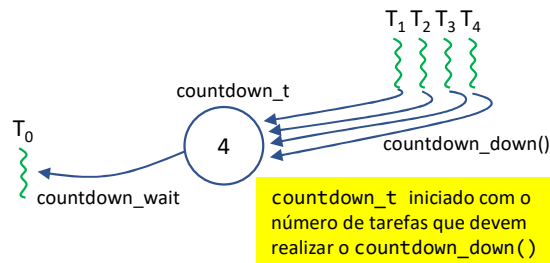


Figura 1 - Exemplificação do funcionamento do mecanismo `countdown_t`

A implementação deve respeitar a seguinte API:

```
typedef struct {
    // a definir com os atributos e mecanismos de sincronismo
    // necessários à sua implementação
} countdown_t;

int countdown_init (countdown_t *cd, int initialValue);
int countdown_destroy (countdown_t *cd);
int countdown_wait (countdown_t *cd);
int countdown_down (countdown_t *cd);
```

- As soluções de código concorrente estruturado com base em múltiplas ações, implica lidar, explicitamente, com a criação e terminação de tarefas para executarem cada uma dessas ações. Como foi discutido nas aulas uma alternativa possível baseia-se na utilização de um *thread pool* (ver Figura 2).

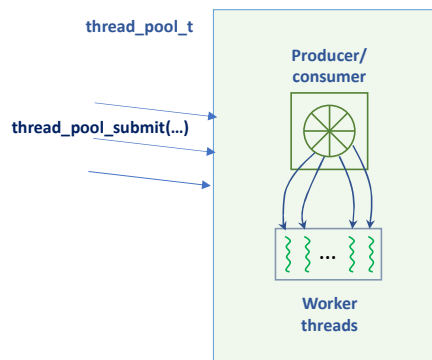


Figura 2 - Estrutura ilustrativa do *thread pool* a desenvolver

Tendo por base a implementação do *thread pool* estudado e implementado nas aulas consolide essa implementação definindo a função `threadpool_destroy()` a ser chamada no fim da utilização do *thread pool* considerando os seguintes pontos para uma terminação graciosa (*graceful termination*):

- O *pool* deve parar de aceitar a submissão de novos trabalhos;
- Os trabalhos, previamente, submetidos devem ser todos executados;
- As tarefas de suporte do *pool* (*worker threads*) devem terminar de forma graciosa após todos os trabalhos terem sido executados. Para as tarefas terminarem devem ser submetidos trabalhos (*work items*) especiais (de terminação) para indicar às tarefas do *pool* que devem terminar;
- A função só retorna depois de todas as *worker threads* terem terminado.

Teste a correção de todas as funcionalidades do seu *thread pool*.

3. Na realização dos trabalhos práticos anteriores desenvolveu variantes, baseadas em múltiplos processos ou tarefas, para determinar o maior, o menor e a soma de todos os elementos num vetor. Realize uma **nova versão baseada no thread pool**. Compare esta versão com a baseada em múltiplos processos, múltiplas tarefas e a versão sequencial.
4. Esta questão é opcional e serve para valorizar o seu trabalho. A resolução pode ser feita totalmente ou parcialmente, ficando a avaliação dependente do contributo adicional apresentado. Este ponto será avaliado com um máximo de 5 valores.

Considere o servidor desenvolvido na 3 questão do 2º trabalho prático (serviço de estatística sobre vetores). Este servidor aceita ligações dos clientes, tanto, através de *sockets* TCP, como, através de *sockets* UNIX. Reformule o servidor, desenvolvido, de forma que passe a incluir os seguintes pontos:

- a. Na abordagem anterior o servidor processava em concorrência os pedidos dos clientes criando uma tarefa por cada ligação. Estas tarefas terminavam após cessar a interação com um cliente. A terminação de todas estas tarefas implica a utilização da primitiva `pthread_join` para que os recursos das tarefas fossem eliminados. Por outro lado, se existisse um elevado número de ligações o servidor acabaria por criar demasiadas tarefas saturando o sistema. Finalmente, o servidor está sempre a despendar tempo na criação e terminação de tarefas.

Altere o servidor de forma que o atendimento dos clientes seja realizado através do **thread pool** que, anteriormente, desenvolveu. Desta forma passam a existir um conjunto fixo de tarefas responsáveis pela interação com os clientes.

- b. Adicione ao servidor o registo de informação estatística relativa ao número de ligações recebidas através de *sockets* no domínio Unix, ligações através de *sockets* no domínio Internet e a dimensão média dos vetores.
- c. Adicione ao servidor uma tarefa **printStatistics** responsável pela apresentação periódica (de segundo em segundo), no terminal do servidor, da informação estatística mantida no servidor.

Esta proposta de trabalho deixa em aberto algumas questões que constituem opções a serem tomadas pelos alunos. Os testes realizados na verificação da correção do trabalho constituem, também, um ponto de avaliação.

II. Indique se as seguintes afirmações são verdadeiras (V) ou falsas (F)

5. Na avaliação a resposta certa será cotada com 100%, resposta errada -25%

a) No contexto de sincronismo indique se as afirmações seguintes são verdadeiras (V) ou falsas (F).

O problema dos filósofos pode ser considerado uma generalização do problema do produtor/consumidor	
Uma solução para o problema dos filósofos será todos começarem por agarrar o seu garfo direito em vez de o esquerdo	
Uma solução pode ser livre de situações de <i>deadlock</i> , mas continuar a apresentar situações de <i>starvation</i> .	
O modelo de múltiplos leitores/escritores é uma otimização da exclusão no acesso a dados.	

b) Considere o modelo de *thread pool* e o mecanismo de barreira utilizados no último trabalho e indique se as afirmações seguintes são verdadeiras (V) ou falsas (F)

Os <i>work items</i> submetidos para um <i>thread pool</i> são, automaticamente, executados de forma sequencial pela mesma tarefa garantido que não existem problemas de concorrência.	
Numa implementação típica de um <i>thread pool</i> as tarefas consumidoras ficam à espera de trabalhos numa fila partilhada que utiliza sincronização para evitar situações de corrida.	
O uso de um <i>thread pool</i> reduz o custo frequente de criação e destruição de tarefas tornando a execução dos trabalhos mais eficiente.	
O mecanismo <code>pthread_barrier_wait()</code> é usado para garantir o acesso exclusivo aos recursos partilhados entre as tarefas.	

c) No contexto de sincronismo indique se as afirmações seguintes são verdadeiras (V) ou falsas (F)

Uma solução para o problema dos filósofos será os filósofos pares começarem por agarrar o seu garfo direito e os ímpares o seu garfo esquerdo.	
O modelo de múltiplos leitores/escritores é um mecanismo de sincronização que permite a leitura simultânea de múltiplas tarefas bloqueando as escritas ou a escrita simultânea de múltiplas tarefas bloqueando as leitoras.	
A utilização da função <code>pthread_cond_wait()</code> do mecanismo de variável de condição obriga que a tarefa esteja em exclusão mútua sendo a exclusão suportada por um <code>pthread_mutex</code> .	
A operação de <code>sem_wait()</code> de um semáforo não bloqueia a tarefa caso o valor do semáforo seja maior que zero.	

d) Considere que a função `handleClient()` processa uma ligação com o cliente indique se as afirmações seguintes são verdadeiras ou falsas.

<pre>int main () { int s = tcp_socket_server_init(PORT); while (1) { int ns = tcp_socket_server_accept(s); handle_client(ns); } return 0; }</pre>	Servidor disponível através de um <i>socket</i> no domínio internet atendendo múltiplos clientes em concorrência.	
	Servidor disponível através de um <i>socket</i> no domínio internet atendendo múltiplos clientes em sequência.	
<pre>void *thHandleClient (void *arg) { int ns = *((int *)arg); handle_client(ns); free(arg); return NULL; } int main () { int s = tcp_socket_server_init(PORT); pthread_t th; while (1) { int ns = tcp_socket_server_accept(s); int *ps = malloc(sizeof(int)); *ps = ns; pthread_create(&th, NULL, thHandleClient, ps); pthread_detach(th); } return 0; }</pre>	Servidor disponível através de um <i>socket</i> no domínio internet atendendo múltiplos clientes em concorrência.	
	Servidor disponível através de um <i>socket</i> no domínio internet atendendo múltiplos clientes em sequência.	

Bom trabalho,
Nuno Oliveira